

PROJECT REPORT

Agentic RAG-Based Citizen Service Advisory Chatbot

GitHub Repository: <https://github.com/abhi6744/Agentic-RAG-Based-Citizen-Service-Advisory-Chatbot>

Submitted by Abhinav G

Roll No: 25225040

Under the guidance of

Dr. Ramesh Chandra Poonia

Dr. Shilpa Shrivastava



School of Sciences

2026-2027

ABSTRACT

Akshaya Advisory is a mobile-first, English-only citizen-service advisory chatbot designed to streamline how residents interact with government services available through Akshaya Centres in Kerala. Navigating bureaucratic requirements often leads to confusion and repeated visits due to missing documentation. To solve this, the application allows users to describe their specific situations in natural language and receive highly structured, actionable guidance.

The Minimum Viable Product (MVP) focuses on three high-friction service domains: Aadhaar Services, Kerala Ration Card Services, and Scholarship Services associated with the National Scholarship Portal. When a citizen asks a question, the system generates a comprehensive response that includes strict eligibility criteria, required document checklists, concrete next steps, exact source citations, and confidence scores, always accompanied by a reminder regarding the need for official verification.

To deliver a seamless cross-platform experience, the frontend is engineered using React Native, Expo, TypeScript, and Expo Router. The robust, asynchronous backend is built with Python, FastAPI, and Uvicorn, utilizing SQLAlchemy and SQLite to securely manage relational data such as conversation history and user feedback.

At the core of the application's intelligence is a specialized Agentic Retrieval-Augmented Generation (RAG) pipeline, which operates far beyond a standard LLM call. Official government documents are securely processed into text chunks and converted into numerical embeddings using the highly optimized sentence-transformers/all-MiniLM-L6-v2 model. A local FAISS vector database enables ultra-fast semantic search across these official records.

When a query is received, the Agentic RAG workflow dynamically orchestrates service classification, conversation context restoration, and query rewriting. The retrieved official context is then passed to the Grok-2 API, which acts as the primary reasoning engine for both structured text generation and vision-based document evaluation. By strictly enforcing citation mapping, confidence scoring, and safe fallback behaviors, the system effectively neutralizes AI hallucinations and ensures that advice remains deeply grounded in official documentation.

Ultimately, Akshaya Advisory serves as an intelligent preparation tool designed to empower citizens before they physically visit an Akshaya Centre. It operates strictly in an advisory capacity; it does not connect to live government databases to submit applications, process payments, guarantee service eligibility, or replace the vital expertise of official Akshaya staff.

TABLE OF CONTENTS

Introduction and Project Objectives	4
Citizen-Service Context and Existing Approaches	6
Requirements and System Analysis	7
Technical Foundations	9
System Design and Architecture	11
Implementation Details	14
Testing and Quality Assurance	16
Results, Limitations and Discussion	18
Screen Shots	19
Conclusion and Future Scope	23
References	23
Appendices	24

CHAPTER 1: INTRODUCTION AND PROJECT OBJECTIVES

1.1 Background

Government services are increasingly delivered through online portals, digital forms, and assisted-service centres. Digital access can reduce waiting time and make public services easier to reach, but only when citizens understand the process. In practice, many citizens know what they need in everyday language but do not know the official name of the service, the department responsible for it, or the documents that must be prepared.

Akshaya Centres play an important role in Kerala by helping citizens use digital public services. A citizen may visit a centre for Aadhaar-related work, ration card changes, scholarship applications, certificates, or other services. When the citizen arrives without the correct documents, the application may be delayed and the person may need to return again. Akshaya Advisory addresses this preparation gap by offering an initial guidance layer through a mobile-friendly chat interface.

1.2 Problem Statement

The information required for a government-service request is often spread across multiple official pages and PDF documents. A user may need to understand the service category, eligibility conditions, acceptable proofs, application steps, and submission location. Search engines can locate pages, but they do not automatically interpret the user's complete situation or produce a verified checklist.

General-purpose language models can understand a question well, but they may produce inaccurate requirements, mix rules from different states, or rely on outdated knowledge. In a government-service context, even a small incorrect detail can result in a failed visit or incomplete application. Therefore, the project requires a system that combines natural-language interaction with strict source grounding.

1.3 Objectives

No.	Objective
1	Build a mobile-first citizen-service advisory chatbot.
2	Support Aadhaar, Kerala Ration Card, and Scholarship guidance.
3	Accept natural-language questions and contextual follow-ups.
4	Retrieve information from official documents.
5	Return structured documents, eligibility, and next steps.
6	Provide citations, confidence, and verification warnings.
7	Support document-image upload and preliminary evaluation.
8	Store conversations and feedback using SQLite.
9	Apply privacy and anti-hallucination safeguards.

1.4 Scope

The MVP is limited to three English-language service areas. Aadhaar support includes enrolment, demographic updates, biometric update information, and document categories. Ration card support includes new applications, modifications, eligibility, and next-step guidance for Kerala. Scholarship support focuses on National Scholarship Portal schemes and related document or eligibility information.

The system is advisory only. It does not submit applications to government portals, process payments, provide legal decisions, or guarantee that an application will be accepted. The user must verify current requirements with the official source or an Akshaya Centre.

1.5 Contributions

- A focused citizen-service assistant instead of a general-purpose chatbot.
- An official-document retrieval layer using local embeddings and FAISS.
- An Agentic Controller for intent detection, context restoration, and retrieval decisions.
- A mobile-first interface with structured response cards.
- Multimodal image input for document-related questions.
- Privacy filters, citation handling, confidence scoring, and safe fallbacks.

CHAPTER 2: CITIZEN-SERVICE CONTEXT AND EXISTING APPROACHES

2.1 Akshaya Centre Context

Akshaya Centres act as assisted digital-service points for citizens. The centre operator may help a user identify an online service, prepare documents, complete a form, or use a government portal. The exact service process remains controlled by the relevant government department. This distinction is important for Akshaya Advisory: the application supports preparation and understanding, but it does not replace the centre operator or the department.

2.2 Existing Approaches

Traditional search is flexible but places the responsibility of interpretation on the user. A citizen may receive a mixture of official pages, older articles, videos, and third-party explanations. Static FAQs are more controlled, but they often require the user to select the correct menu or phrase the question using expected keywords. General AI assistants understand informal language but may not be tied to Kerala-specific and current official documents.

Approach	Strength	Limitation
Search engine	Finds a wide range of pages	User must compare and interpret results
Static FAQ	Controlled and predictable	Rigid navigation and limited follow-up
General chatbot	Natural conversation	May hallucinate or lack local grounding
Akshaya Advisory	Natural input plus official retrieval	Limited to the current three-service knowledge base

2.3 Design Position

Akshaya Advisory does not attempt to replace search engines. Its purpose is narrower: reduce the effort required to translate a citizen's situation into a supported service, retrieve relevant official text, and present it as a structured preparation guide. The application is most useful when the question is unclear, involves a follow-up, or requires the user to compare documents and next steps.

2.4 User Journey

- The citizen describes the issue in everyday language.
- The system identifies the likely supported service.
- The system searches official document chunks.
- The response is divided into summary, eligibility, documents, next steps, and source information.
- The citizen uses the guidance to prepare for the next interaction with an Akshaya Centre or official portal.

2.5 User Personas

Persona	Need
First-time applicant	Needs to understand the service and documents before visiting a centre.
Student	Needs scholarship eligibility and application preparation guidance.
Family applicant	Needs ration card eligibility, member changes, or submission guidance.
Identity-update user	Needs Aadhaar update document categories and next steps.
Akshaya operator	May use the assistant as a preliminary information tool, not as final authority.

CHAPTER 3: REQUIREMENTS AND SYSTEM ANALYSIS

3.1 Stakeholders

The primary stakeholder is the citizen. The citizen interacts with the mobile interface, asks questions, reviews structured advice, and decides whether to visit an Akshaya Centre or official portal. The project maintainer manages source documents, indexing, configuration, and application updates. Government authorities remain the source of final rules and procedures.

3.2 Functional Requirements

ID	Requirement
FR01	Submit a text question from the mobile interface.
FR02	Classify a question into Aadhaar, ration card, scholarship, or unsupported.
FR03	Retrieve relevant official document chunks.
FR04	Generate a structured answer.
FR05	Display eligibility conditions when supported.
FR06	Display documents when supported.
FR07	Display next steps and location guidance when supported.
FR08	Understand contextual follow-up questions.
FR09	Display citations, confidence, and official verification.
FR10	Upload an image with an optional question.
FR11	Record conversations and retrieve history.
FR12	Record helpful or improvement feedback.
FR13	Return a safe fallback when evidence is insufficient.

3.3 Non-Functional Requirements

Category	Requirement
Usability	The user should be able to ask a question without navigating complex menus.
Readability	Responses should use clear sections, dark text, and adequate spacing.
Privacy	Sensitive values should not be logged or repeated.
Security	Secrets must be stored outside source code.
Performance	The small local index should support fast retrieval.
Maintainability	Frontend, backend, ingestion, and services should be separated.
Traceability	Claims should be linked to source metadata.
Reliability	Errors should be handled without crashing the application.
Extensibility	New services and documents should be addable later.

3.4 Use Cases

Use case	Description
Service discovery	User describes a problem and the system identifies a likely supported service.
Document guidance	User asks what documents to carry or upload.
Eligibility guidance	User asks whether a condition or scheme may apply.
Next-step guidance	User asks where to go or what to do next.
Image evaluation	User submits a document image for preliminary interpretation.
Source review	User opens the official source references.
History	User revisits previous conversations.
Feedback	User evaluates the usefulness of a response.

3.5 Constraints

- Only English is supported in the current version.
- Only three service domains are indexed.
- The application depends on the availability of the backend and configured LLM provider.
- Government rules may change after documents are ingested.
- Image quality affects vision-based evaluation.
- Final decisions remain with the relevant authority.

CHAPTER 4: TECHNICAL FOUNDATIONS

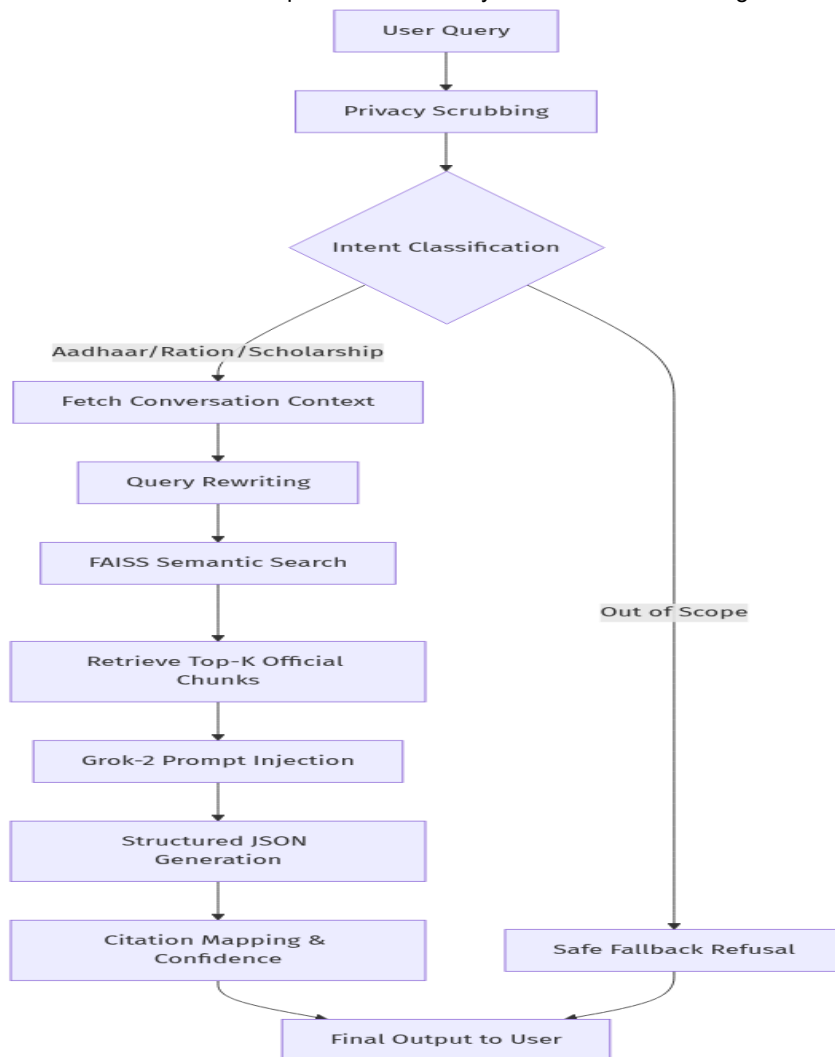
4.1 Retrieval-Augmented Generation

RAG separates knowledge retrieval from answer generation. The application first searches its indexed official content, then supplies the most relevant passages to the language model. This reduces the need for the model to rely on general memory. In this project, the retrieved passages are also used to attach citations and calculate a confidence signal.

4.2 Agentic RAG

The term Agentic RAG refers to the decision-making behavior around retrieval. The Agentic Controller is responsible for more than calling a vector search function. It identifies intent, carries conversation context, rewrites incomplete follow-ups, chooses a service-specific retrieval path, checks whether evidence is sufficient, and decides whether a safe fallback is needed.

For example, the question “Where should I go?” is incomplete by itself. If the previous question was about an Aadhaar name update, the controller combines the previous context with the new message and searches for Aadhaar-related next-step information. This improves usability without allowing the model to invent an answer.



4.3 Embeddings

The all-MiniLM-L6-v2 sentence-transformer maps text into 384-dimensional vectors. Similar meanings are represented by nearby vectors. The same embedding model is used for document chunks and user queries so that semantic similarity is meaningful.

4.4 FAISS

FAISS is used as a local vector index. It avoids an external vector database service and is suitable for a small MVP knowledge base. The index is accompanied by metadata that maps vector identifiers to chunk text, service category, source title, authority, URL, and retrieval date.

4.5 SQLite and SQLAlchemy

SQLite stores relational application data. SQLAlchemy provides the database mapping and query layer. This separation prevents temporary user messages or uploaded images from being confused with official source chunks. SQLite is appropriate for the MVP because it does not require a separate database server.

4.6 Grok-2

Grok-2 is the documented LLM provider for text generation and vision processing. The model receives a controlled prompt, the user query, relevant retrieved context, and conversation context. It is instructed to return structured information and avoid unsupported government facts.

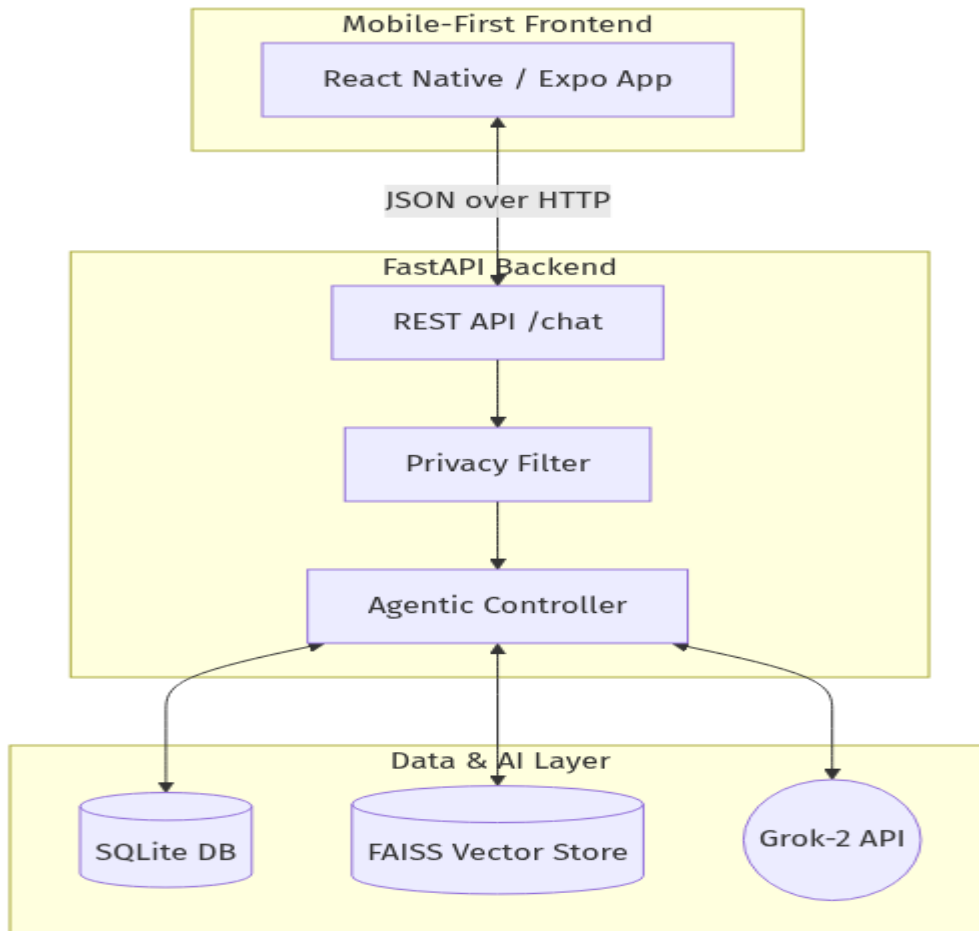
4.7 Privacy and Safety Principles

- Do not place API keys in source code or public repositories.
- Do not upload unnecessary Aadhaar numbers, OTPs, passwords, bank details, or biometric data.
- Do not store user images in FAISS.
- Delete temporary image files after evaluation where configured.
- Use official verification warnings for government requirements.
- Return an insufficient-information response when retrieval is weak.

CHAPTER 5: SYSTEM DESIGN AND ARCHITECTURE

5.1 High-Level Architecture

The frontend and backend communicate through REST-style HTTP requests. The frontend collects text, optional images, and conversation information. The backend validates the request, restores context, retrieves official information, calls the configured model, validates the response, stores the turn, and returns structured data.



5.2 Query Processing Pipeline

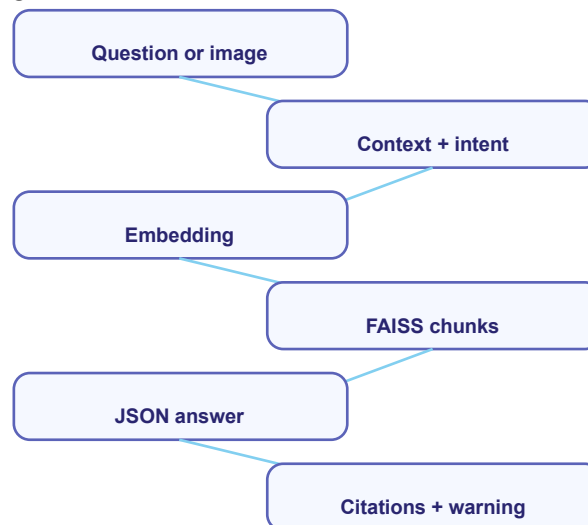
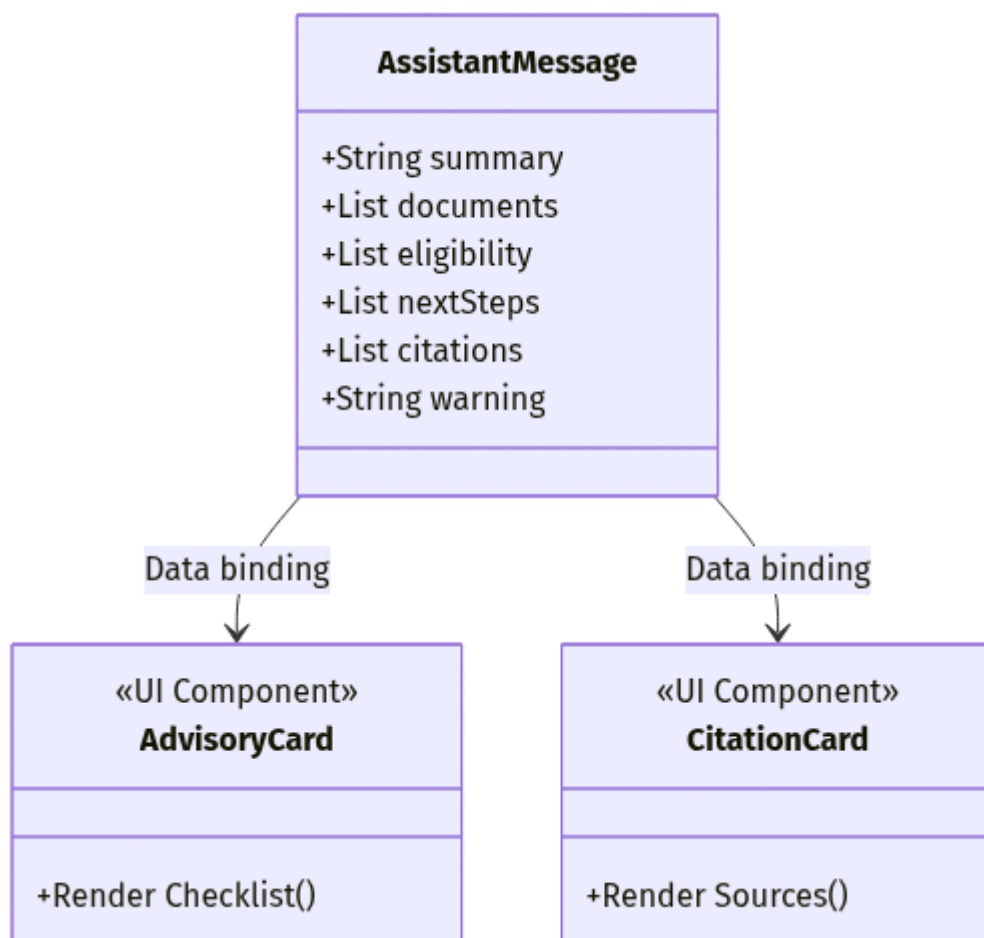


Figure 2. Agentic RAG query pipeline

5.3 Frontend Architecture

The mobile interface uses Expo and React Native. Expo Router manages application routes. The main chat screen contains the header, welcome content, message list, attachment controls, text composer, structured answer cards, source information, confidence status, and feedback controls. A navigation drawer provides access to supported service information, account and privacy information, about content, and history.



5.4 Visual Design

The UI uses a light blue and white design to create a calm and readable experience. The main colors include primary blue #5B63B8, deep blue #29256F, light blue #EAF3FF, pale background #F8FAFF, white surfaces, and dark navy text #20233D. Rounded cards group related information without creating a dense form-like interface.

5.5 Backend Architecture

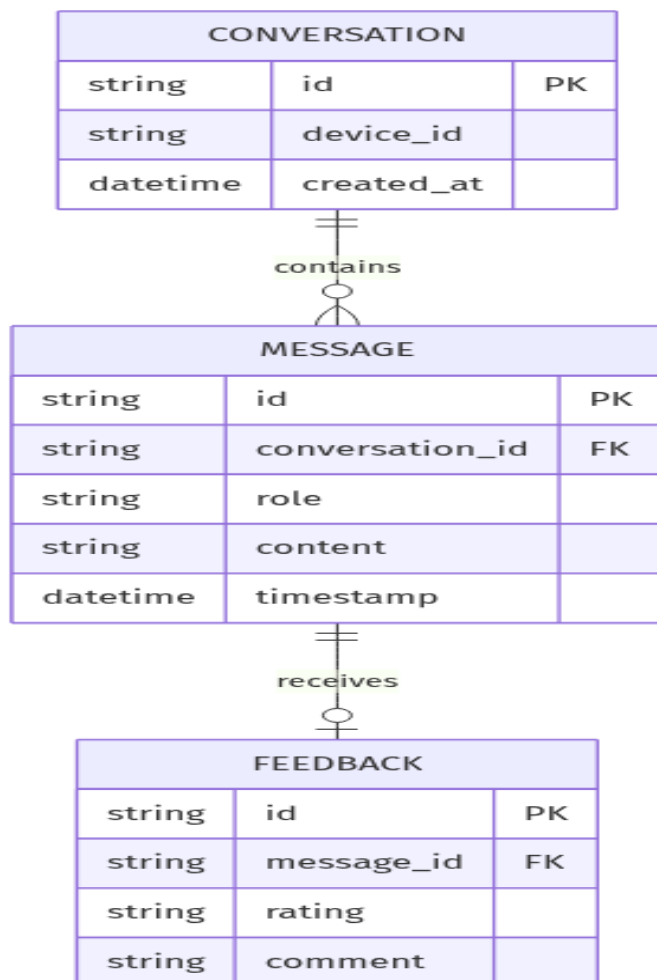
The FastAPI backend is divided into routes, schemas, models, database utilities, ingestion scripts, and services. The service layer includes privacy filtering, intent classification, retrieval, agentic control, LLM calls, citation validation, and confidence scoring. This separation allows the project to test the retrieval path independently from the user interface.

5.6 Data Flow

A normal text request travels from the composer to `POST /chat`. A request with an image uses multipart form data. The backend validates the upload, stores it temporarily, sends it to the vision model when configured, and deletes the temporary file after processing. The response is returned as structured JSON rather than an unstructured paragraph.

5.7 Database Design

Table	Purpose
Conversations	Tracks the conversation identifier, device reference, and timestamps.
Messages	Stores user and assistant turns with role and conversation relation.
Feedback	Stores helpful or improvement feedback and optional comments.
Message sources	Associates official source metadata with an assistant response.
Document registry	Tracks documents used in the knowledge base.
Query audit log	Records retrieval and validation information where implemented.



CHAPTER 6: IMPLEMENTATION DETAILS

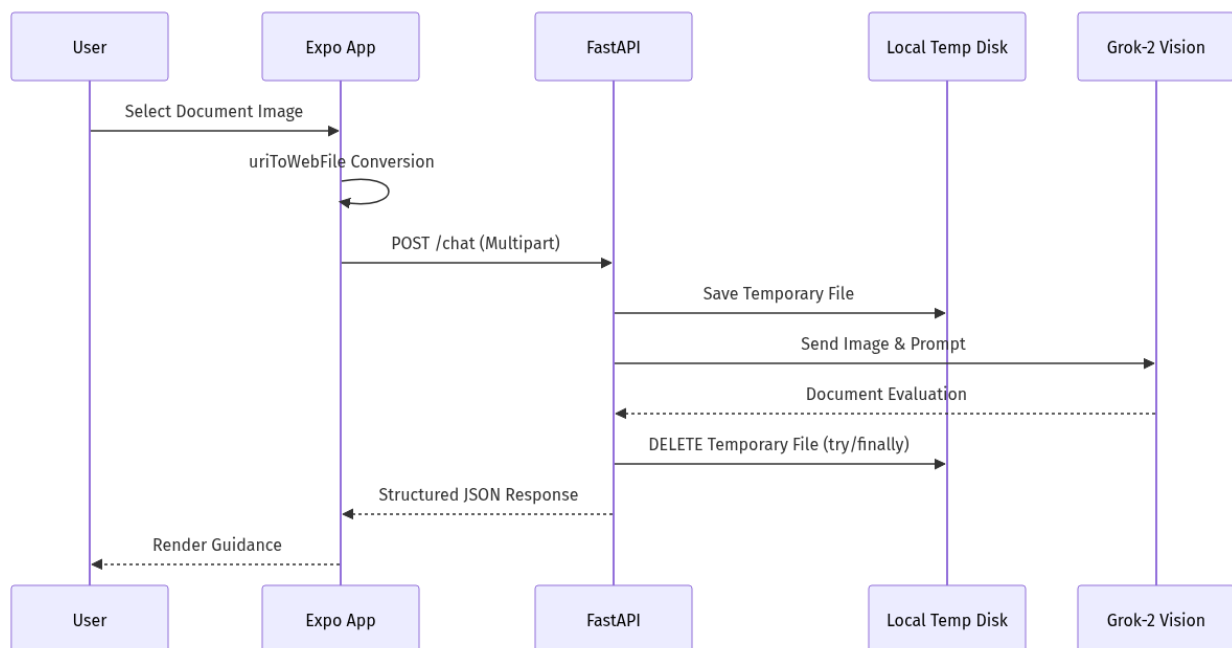
6.1 Frontend Implementation

The frontend was implemented as a cross-platform React Native application using Expo and TypeScript. The chat screen was kept deliberately simple. The user does not need to select a service category before asking a question. Suggested questions help new users understand the supported scope. The application renders user messages on one side and assistant messages on the other, while structured response sections are displayed below the answer.

6.2 Image Upload Implementation

The image workflow required separate handling for browser and native environments. Expo Image Picker returns a URI that may represent a browser object URL on the web. The `uriToWebFile` utility converts that URI into a standard File object before the browser sends it through FormData. On native platforms, the image URI is sent in the platform-supported format.

The `DocumentImage` component uses a web-compatible image renderer for browser URLs and the React Native Image component for native paths. This prevents broken-image errors when a browser-generated blob URL is displayed. The message retains its image reference after the composer is cleared so the document remains visible in the conversation.



6.3 Backend Implementation

The FastAPI application exposes the chat operation and supporting endpoints. Pydantic schemas validate incoming requests and structure outgoing responses. The LLM response is expected to contain summary, eligibility, documents, next steps, where-to-go guidance, warning text, citations, and confidence information. Missing fields are normalized so the frontend can render them safely.

6.4 Ingestion Implementation

The document ingestion process reads official PDFs, extracts their text, cleans unnecessary formatting, creates chunks, and adds source metadata. Embeddings are generated locally. The resulting vectors are stored in FAISS and the metadata is stored separately so that every retrieved vector can be translated back into readable source information.

6.5 Context Memory

When a new conversation starts, the system classifies the initial question. For later messages, the backend loads recent turns from SQLite. Short questions such as “Where do I submit this?” are combined with the previous service context before retrieval. This makes the chat feel conversational while retaining the strict source requirement.

6.6 Structured Response Rendering

The response is not rendered as arbitrary HTML. The backend returns plain-text arrays, and the frontend renders them using native text and list components. Documents appear as bullet items, while recommended next steps appear as numbered items. This avoids exposing markup such as HTML list tags to the user and improves readability on a small screen.

6.7 Error Handling

The implementation includes handling for invalid model output, missing configuration, backend connection problems, invalid image files, and low-confidence retrieval. Errors are converted into structured responses so that the mobile application remains usable instead of displaying an unhandled exception.

CHAPTER 7: TESTING AND QUALITY ASSURANCE

7.1 Testing Strategy

Testing was performed at several levels. Backend tests verify the API and service logic. Retrieval tests verify that the correct service documents are returned. Frontend tests verify screens, interactions, image rendering, and error states. Integration tests verify the complete path between the mobile client, FastAPI backend, FAISS index, database, and LLM provider.

7.2 Functional Test Cases

ID	Test	Expected result
FT01	Open the application	The blue-white chat interface loads.
FT02	Ask an Aadhaar question	Aadhaar is identified and grounded guidance is displayed.
FT03	Ask a ration card question	Ration card information is retrieved and structured.
FT04	Ask a scholarship question	Scholarship information is returned with verification guidance.
FT05	Ask "Where should I go?" after a service question	Previous service context is retained.
FT06	Ask an unrelated question	The system responds politely without inventing service facts.
FT07	Open history	Actual stored conversations are listed.
FT08	Submit helpful feedback	Confirmation appears and the record is stored.
FT09	Submit improvement feedback	Optional comment can be recorded.
FT10	Upload a document image	The image is previewed and sent correctly.

7.3 Retrieval Tests

Test	Expected outcome
Aadhaar classification	Aadhaar chunks are selected.
Ration card classification	Kerala ration card chunks are selected.
Scholarship classification	Scholarship chunks are selected.
Contextual query	The previous service is used to rewrite the query.
Low similarity	The system refuses to guess.
Citation mapping	Source title and URL are attached.

7.4 Image Tests

Image testing covered selection, preview, removal, reselection, multipart upload, backend validation, temporary processing, message rendering, vision evaluation, and failure handling. A valid image should appear in the user message.

An invalid or unclear image should produce a clear message rather than a broken image or unsupported document claim.

7.5 Security Tests

- No secret should be exposed in frontend source.
- The .env file must remain outside Git.
- Uploaded images must not enter the vector index.

- Sensitive values should not appear in model responses or logs.
- Temporary files should be removed after processing.
- Server errors should not expose internal tracebacks to citizens.

7.6 Usability Tests

The interface was reviewed on browser and mobile-sized layouts. The main checks included readable text, adequate spacing, bottom composer behavior, scrolling for long responses, source link visibility, image card size, feedback interaction, and history navigation.

CHAPTER 8: RESULTS, LIMITATIONS AND DISCUSSION

8.1 Implemented Results

The completed implementation demonstrates an end-to-end full-stack application. A citizen can open the mobile interface, enter a question, receive an answer from the backend, and view the response in structured sections. The backend can connect the user query to retrieved official content and return source-aware guidance.

Area	Result
User interface	Mobile-first chat UI with light blue and white theme.
Service scope	Aadhaar, Kerala Ration Card, and Scholarship.
Backend	FastAPI API with validation and service modules.
Structured storage	SQLite for conversations, messages, and feedback.
Semantic retrieval	all-MiniLM-L6-v2 embeddings and FAISS.
Language model	Grok-2 for text and configured vision processing.
Context memory	Follow-up questions use conversation history.
Image support	Optional document upload and evaluation path.
Safety	Citations, confidence, privacy filter, verification warning.

8.2 Engineering Challenges

The project required solving several practical problems. One issue involved model output that did not follow the expected JSON shape. Defensive error handling and logging were needed so that an invalid response did not crash the server. Another issue involved web and Android networking. A browser can reach localhost directly, but an Android emulator uses a different loopback address, so the API routing required environment-aware configuration.

Image rendering created another cross-platform challenge. Browser object URLs could not always be rendered using the same component used on native platforms. A platform-aware `DocumentImage` component was introduced to handle web and native paths separately. These problems demonstrate why cross-platform development requires testing in the actual environments where the app will run.

8.3 Accuracy and Grounding Discussion

The project uses strong safeguards to reduce unsupported answers, but these safeguards should not be described as a mathematical guarantee of zero hallucination. The result depends on document quality, retrieval quality, prompt behavior, citation validation, and the current configuration of the language model. The correct product claim is that the system is designed to refuse or warn when evidence is insufficient and to ground supported answers in official chunks.

8.4 Limitations

- The knowledge base covers only three service domains.
- The application is English-only.
- Documents may become outdated when government rules change.
- The system does not submit applications or process payments.
- Vision results depend on image clarity and model availability.
- A local FAISS index requires a rebuild when source documents change.
- SQLite is suitable for the MVP but may require migration for a large multi-user deployment.
- The system requires stronger authentication and monitoring before public production use.

8.5 Practical Value

The application provides value by reducing the initial information burden on citizens. It can help a user understand what kind of service may be relevant, prepare likely documents, and identify the need for official confirmation. Its value is therefore measured by clarity and preparation support, not by replacing the official application process.

SCREEN SHOTS

**Akshaya Advisory**
Citizen-service assistant



How can we help today?

Describe your issue in your own words - Aadhaar, ration card, or scholarship. You can also attach a photo of a document.

 Clear advisory guidance before you visit an Akshaya centre.

Try asking

 What documents do I need for a new ration card in Kerala? >

 Am I eligible for a National Scholarship, and what documents are needed? >

 My name in Aadhaar doesn't match my certificate - how do I update it? >

Avoid uploading photos showing your Aadhaar number, OTP, or other sensitive numbers in full view.

 Attach

 Voice

Describe your issue or question...

 Send



What documents do I need for a new ration card in Kerala?



To apply for a new ration card in Kerala, the head of the household must submit an application in Form A-1 to A-12 along with a photograph and a certificate from the competent rationing authority confirming that the applicant is not included in any existing ration card or has been officially excluded from one.



Official Verification Required

Please verify current requirements with your nearest Akshaya Centre or the official government portal.



Identified Service

Ration Card Services



Documents and eligibility

Documents to prepare

- Application in Form A-1 to A-12 as per Annexure-I.
- Photograph of the head of the household.
- Certificate from the competent rationing authority confirming non-inclusion or official exclusion from any existing ration card.

Eligibility or important conditions

- The applicant must be the head of the household.
- The applicant must produce a certificate from the competent rationing authority of the rationing area concerned, whether within the State or outside the State, stating that he is not included in a ration card or

Avoid uploading photos showing your Aadhaar number, OTP, or other sensitive numbers in full view.



Attach



Voice

Describe your issue or question...



Send



Recommended next steps

1. Present the application directly to the Taluk Supply Officer of the rationing area concerned or submit it online.
2. The Taluk Supply Officer will conduct necessary enquiries and process the application for issuance of the ration card.

Where to apply or seek help

- Taluk Supply Officer office of the rationing area concerned.
- Online portal, if notified by the Commissioner of Civil Supplies for exclusive online service delivery.

Sources & Confidence (77 - HIGH)

[Kerala Ration Card Rules \(KRO\) \(Kerala Civil Supplies Department\)](#)

[Kerala Revenue Department Services \(Kerala Revenue Department\)](#)

 Play audio

 Pause

 Stop

Was this guidance useful?



Helpful



Not helpful

Avoid uploading photos showing your Aadhaar number, OTP, or other sensitive numbers in full view.



Attach

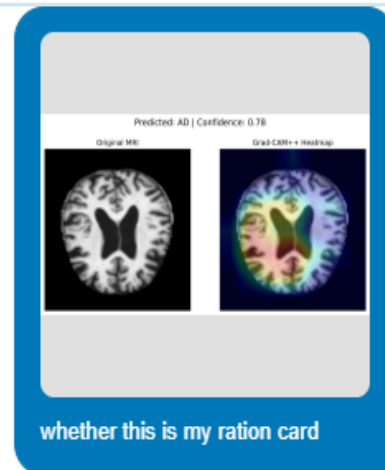


Voice

Describe your issue or question...



Send



The provided image is a medical brain MRI scan with an Alzheimer's Disease prediction heatmap, not a ration card.



Official Verification Required

Please verify current requirements with your nearest Akshaya Centre or the official government portal.



Identified Service

Ration Card Services



Documents and eligibility

The retrieved official sources did not provide a detailed checklist for this specific question.

Avoid uploading photos showing your Aadhaar number, OTP, or other sensitive numbers in full view.



Attach



Voice

Describe your issue or question...



Send

CHAPTER 9: CONCLUSION AND FUTURE SCOPE

9.1 Conclusion

Akshaya Advisory demonstrates a complete mobile and backend workflow for source-aware citizen-service guidance. The project combines a simple mobile interface with an Agentic RAG pipeline that classifies the query, retrieves official evidence, preserves conversation context, generates structured output, and attaches confidence and verification information.

The project's central design decision is to treat the language model as an assistant over official evidence rather than as an independent authority. This decision influences the document pipeline, vector index, response schema, citation layer, privacy filter, and fallback behavior. It also provides a clear path for evaluation because each response can be inspected against the source chunks used to produce it.

9.2 Future Scope

- Add Malayalam support after completing English evaluation.
- Expand the knowledge base to additional Akshaya services.
- Add an administrator panel for source-document updates.
- Monitor document freshness and rebuild embeddings automatically.
- Improve OCR and field-level document comparison.
- Add a secure centre locator using verified location data.
- Introduce stronger authentication and user data isolation.
- Deploy FAISS and SQLite replacements for larger production workloads where necessary.
- Create an evaluation dataset with real but anonymized service questions.
- Add voice input and improved accessibility support.

9.3 Final Statement

Akshaya Advisory provides a practical foundation for helping citizens understand selected government services before visiting an Akshaya Centre. Its mobile interface, local semantic retrieval, conversation context, document-image pathway, citations, and safety warnings work together to provide guidance that is clearer than a collection of disconnected links. The application remains an advisory system, and official verification remains essential for every final decision.

REFERENCES

1. Akshaya Kerala official service portal and service catalogue.
2. Kerala e-District official service documentation.
3. Kerala Civil Supplies Department official ration card documentation.
4. Unique Identification Authority of India official enrolment and update documentation.
5. National Scholarship Portal official guidance and FAQs.
6. FastAPI official documentation.
7. Expo and React Native official documentation.
8. FAISS documentation for similarity search.
9. Sentence Transformers documentation for all-MiniLM-L6-v2.
10. SQLite and SQLAlchemy documentation.
11. Grok API documentation for text and vision capabilities.

APPENDIX A: TECHNOLOGY STACK

Layer	Technology	Role
Frontend	React Native, Expo, TypeScript, Expo Router	Mobile interface and navigation
Backend	Python, FastAPI, Uvicorn	API and application services
Database	SQLite, SQLAlchemy	Relational application data
Embeddings	all-MiniLM-L6-v2	384-dimensional semantic vectors
Vector search	FAISS	Local similarity retrieval
LLM	Grok-2 latest	Text and vision generation
Image input	Expo Image Picker, FormData	Document-image upload
Configuration	python-dotenv	Environment-based secrets

APPENDIX B: API ENDPOINTS

Method	Endpoint	Purpose
GET	/health	Backend and index status.
POST	/chat	Text and optional image question.
POST	/feedback	Helpful or improvement feedback.
GET	/services	Supported service list.
GET	/conversations	Conversation history list.
GET	/conversations/{id}	Messages for one conversation.

APPENDIX C: DATABASE AND SOURCE METADATA

Record	Important fields
Conversation	id, device reference, title, created time
Message	conversation id, role, text, service, confidence, timestamp
Feedback	message id, type, optional comment, timestamp
Source	title, URL, authority, retrieved date, chunk id
Chunk	service category, text, source metadata, vector id

A source record is retained with each retrieved chunk so that the response can identify the official document that supported the answer. Personal uploaded images are not treated as source documents.

APPENDIX D: TEST CASES

ID	Input	Expected result
TC01	Aadhaar document question	Grounded Aadhaar response.
TC02	Ration card document question	Grounded Kerala ration card response.
TC03	Scholarship eligibility question	Grounded scholarship response.
TC04	Where should I go? after Aadhaar question	Aadhaar contextual next-step answer.
TC05	Unrelated factual question	Polite scope response.
TC06	Valid image and document question	Image remains visible and backend evaluates it.
TC07	Unclear image	Safe request for a clearer image.
TC08	Feedback	UI confirmation and SQLite record.
TC09	History	Real conversations appear and reopen.